

DIGITAL CERTIFICATE VERIFICATION SYSTEM

Java Project Documentation

Name: Mahek Yadav

Roll no.: 150096725142

Cohort: Sam Altman

1. Abstract

The Digital Certificate Verification System is a Java Swing-based desktop application used to generate, manage and verify digital certificates. It provides certificate generation, verification, searching, updating, deletion, sorting and reporting features.

The system uses Java concepts such as classes and objects, constructors, interfaces, StringBuilder, ArrayList, LinkedList, HashMap, TreeMap, CRUD operations, searching, sorting, exception handling and validation.

2. Introduction

Digital certificates are commonly used to prove completion of courses or participation in events. Managing certificates manually can make searching, verification and record maintenance difficult.

This project provides a simple desktop application where certificates can be generated with unique IDs and later searched or verified using the stored information.

3. Problem Statement

To develop a Java-based system that can generate unique digital certificates and provide easy certificate management and verification through a graphical user interface.

The system should support:

- Certificate generation
- Certificate verification
- Searching
- Updating
- Deletion
- Sorting
- Issuance history
- Duplicate prevention

4. Objectives

- To generate unique certificate IDs.
- To store and manage certificate records.

- To verify certificates using their ID.
- To prevent duplicate certificates.
- To implement CRUD operations.
- To provide searching and sorting.
- To maintain certificate history.
- To provide a simple graphical user interface.

5. Technologies Used

Technology	Purpose
Java	Main programming language
Java Swing	Graphical User Interface
ArrayList	Store certificates
LinkedList	Store issuance history
HashMap	Search certificates by ID
TreeMap	Store certificates according to date
StringBuilder	Generate IDs and reports

6. Concepts Used

Classes and Objects

Classes such as `Recipient`, `Course`, `Certificate` and `Verifier` are used to represent different parts of the system.

Constructors

Constructors initialize objects with required information.

Interface

The `Verifiable` interface contains the `verify()` method, which is implemented by the `Certificate` class.

ArrayList

Stores all generated certificate objects.

LinkedList

Maintains the issuance history.

HashMap

Stores certificates using the certificate ID as the key for quick searching.

TreeMap

Groups certificates according to their issue date.

StringBuilder

Used to generate certificate IDs and construct reports.

CRUD

- **Create:** Generate certificate
- **Read:** Search and display certificate
- **Update:** Modify recipient details
- **Delete:** Remove certificate

Searching

Certificates can be searched using:

- Certificate ID
- Recipient ID
- Recipient Name

Sorting

Certificates can be sorted:

- By issue date
- By course name alphabetically

7. System Requirements

Hardware

- Laptop
- Minimum 4 GB RAM

Software

- Java JDK
- VS Code
- Operating system supporting Java

8. Main Modules

8.1 Certificate Generation

The user enters recipient and course details. The system validates the information, checks for duplicates and generates a unique certificate ID.

Example:

CERT-JAVA101-001

8.2 Certificate Verification

The certificate ID is searched in the `HashMap`. If the certificate exists and is valid, its details are displayed.

8.3 Search

The system searches certificates using certificate ID, recipient ID or recipient name.

8.4 Update

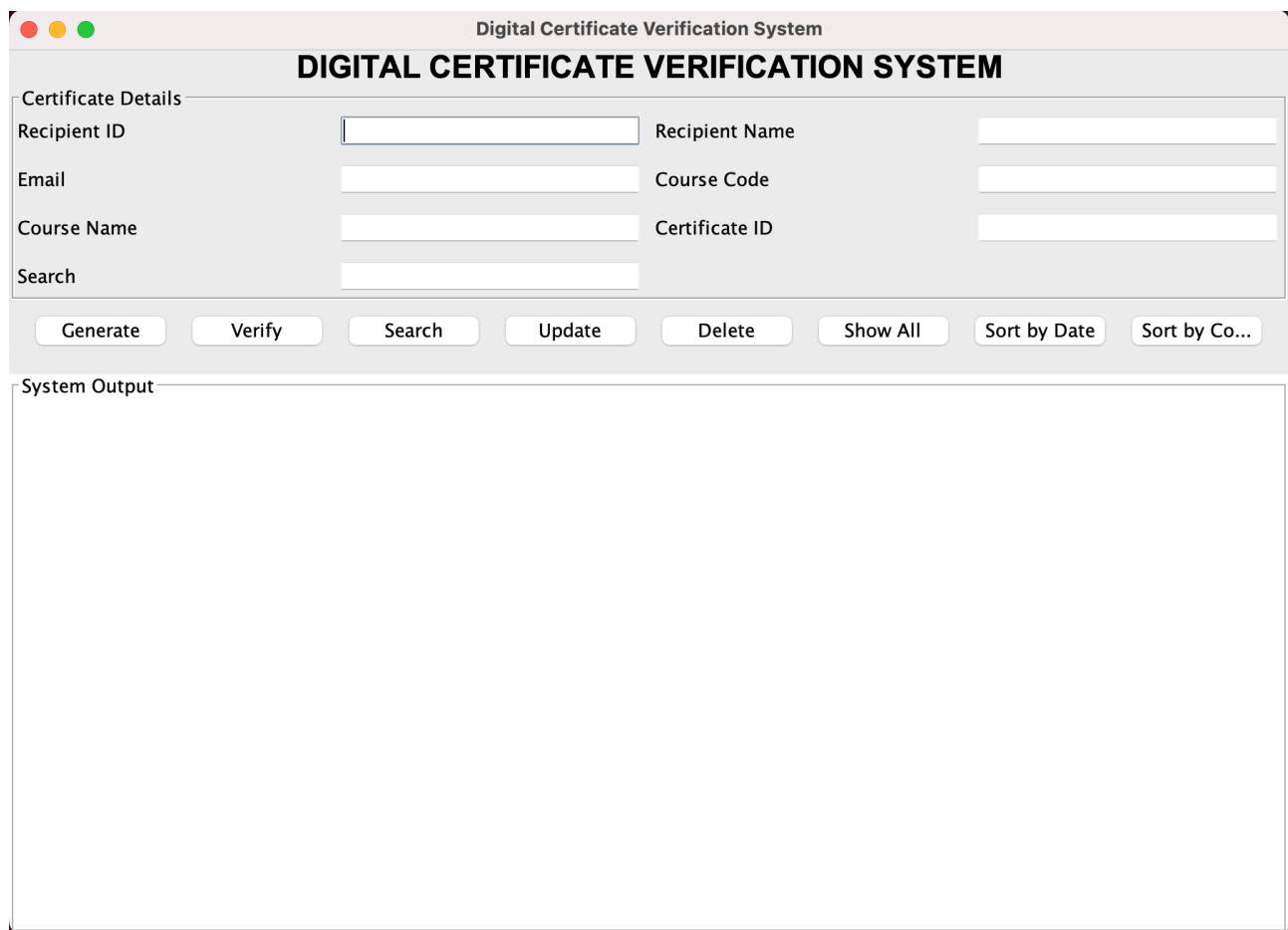
The user selects an existing certificate and enters new recipient name and email. The certificate record is then updated.

8.5 Delete

The selected certificate is removed after confirmation.

8.6 Sorting

Certificates can be sorted by issue date or course name.



The screenshot displays a web application titled "Digital Certificate Verification System". The interface is divided into two main sections. The top section, labeled "Certificate Details", contains a form with six input fields: "Recipient ID", "Recipient Name", "Email", "Course Code", "Course Name", and "Certificate ID". Below these fields is a "Search" button. The bottom section, labeled "System Output", is a large, empty text area. A row of buttons is positioned between the input fields and the output area, including "Generate", "Verify", "Search", "Update", "Delete", "Show All", "Sort by Date", and "Sort by Co...".

9. System Flow

```
User
  ↓
Enter Certificate Details
  ↓
Validation
  ↓
Check Duplicate
  ↓
Generate Certificate
  ↓
Store Certificate
  ↓
Search / Verify / Update / Delete / Sort
  ↓
Display Output
```

10. Data Structures

Data Structure	Use
ArrayList	Stores certificates
LinkedList	Stores issuance history
HashMap	Certificate ID based searching
TreeMap	Date-based certificate grouping
Array	Stores GUI buttons for common settings

11. Exception Handling and Validation

The system validates user input before generating a certificate.

It checks:

- Required fields are not empty.
- Email contains @.
- Course code follows the required format, such as `JAVA101`.
- Duplicate certificates are not allowed.
- Certificate must exist before update or deletion.

Errors are handled using `try-catch` and displayed using Swing error messages.

12. User Interface

The application contains:

- Recipient ID field
- Recipient Name field
- Email field
- Course Code field
- Course Name field
- Certificate ID field
- Search field
- Generate button
- Verify button
- Search button
- Update button
- Delete button
- Show All button
- Sort by Date button
- Sort by Course button
- Large System Output area

13. Advantages

- Simple and user-friendly interface.
- Automatic certificate ID generation.
- Prevents duplicate certificates.
- Easy certificate verification.
- Supports searching and sorting.
- Demonstrates multiple Java concepts.
- Provides basic certificate management in one application.

14. Limitations

- Data is stored only during program execution.
- No database is used.
- No online verification.
- No login/authentication system.
- No PDF certificate generation.

15. Future Enhancements

- Add database connectivity.
- Add user login and authentication.
- Add online certificate verification.
- Generate certificates as PDF files.
- Add QR-code based verification.
- Add cloud storage.

16. Conclusion

The Digital Certificate Verification System successfully provides a simple solution for generating, managing and verifying digital certificates. It implements CRUD operations, searching, sorting, validation and verification using Java and Swing.

The project also demonstrates important Java concepts including classes, objects, interfaces, constructors, collections, StringBuilder, exception handling and GUI programming.